Product-Company Comeback Plan: Weeks 1 to 6

Prepared for Akash by Claude. Start: Mon Oct 26, 2026. End: Sun Dec 6, 2026. This is Phase 1 of a 30-week plan: rebuild the base.

What you will have by Dec 6

- 45 Neetcode problems solved and revisited (Arrays and Hashing, Two Pointers, Sliding Window, Stack, Binary Search, Linked List)
- Refreshed Java collections, Java 8 to 21 features, concurrency, JVM and Spring internals
- A quantified one-page resume, updated LinkedIn and GitHub, and a referral contact list of 10+ people
- An honest checkpoint result that tells you if you are ready for Phase 2 (trees, heaps, graphs, DP)

Ground rules

- Time: weekdays 2.5 hours, weekends 5 to 6 hours, same slot every day. Day 1 (today) is about 3 hours because of setup.
- Weekday split: DSA 90 min (new problem 60, revisit 15, logging 15), Java or Spring 45 min, recap 15 min.
- Weekend split: DSA 3 h, Java 1.5 h, career task 1 h, weekly review 30 min.
- Language: Java 21 only. Turn AI autocomplete OFF in your IDE while practicing.
- Missed a day: do not double up. Push that day's new problem forward. Never skip the Sunday review. If a festival week eats days, shift the whole plan by up to 3 days.
- Worst-day rule: do at least one 15-minute revisit so the streak survives.
- Weeks 1 and 2 have 1 new problem per day on purpose. Depth beats volume while the base is weak.

The 10-step protocol for every problem

1. Read it twice and restate it in your own words.
2. Write 3 examples including an edge case (empty input, single element, duplicates, negatives).
3. Say the brute force and its time and space complexity out loud.

4. Find the bottleneck and ask which pattern removes it (hash map, two pointers, window, stack, binary search).
5. Code it in Java with no autocomplete.
6. Dry-run your examples line by line.
7. Stuck for 25 minutes? Read or watch only up to the key idea, close it, then code it yourself.
8. After it passes, close the solution and re-code from scratch.
9. Log it in the tracker: pattern, one-line insight, mistake, time, solved alone (Y/N).
10. Schedule revisits at +1, +3, +7 and +21 days.

Tracker columns (Google Sheet)

Date | LC number and name | Difficulty | Pattern | Minutes taken | Solved alone (Y/N) | Key insight | Mistake made | Revisit dates | Confidence 1 to 5

Week 1 (Oct 26 to Nov 1): Setup, Big-O and the hash map pattern

Goal: working system, Java collections fluency, 7 new problems.

Mon Oct 26 (Day 1, about 3 h). Setup: LeetCode account, GitHub repo dsa-java with one folder per pattern, tracker sheet, IntelliJ with AI completion off, Neetcode 150 bookmarked. Learn Big-O (one 1-hour video): O(1), O(log n), O(n), O(n log n), O(n^2), space complexity. New: Contains Duplicate (LC 217): brute force O(n^2), sort O(n log n), HashSet O(n). Java: write a cheat sheet of operation complexity for ArrayList, LinkedList, HashMap, HashSet, TreeMap, ArrayDeque, PriorityQueue.

Tue Oct 27. New: Valid Anagram (242): int[26] frequency array vs HashMap. Revisit: 217 from scratch. Java: HashMap and HashSet API drill (getOrDefault, merge, computeIfAbsent, putIfAbsent, entrySet loops); write 10 mini snippets.

Wed Oct 28. New: Two Sum (1): one pass, HashMap of value to index. Revisit: 242. Java: equals and hashCode contract; use a custom class as a HashMap key; demo the mutable-key bug.

Thu Oct 29. New: Group Anagrams (49): key = sorted string, then the 26-count key. Revisit: 1. Java: HashMap internals 1: bucket array, hash spreading (h ^ h>>>16), index = (n-1) & hash, load factor 0.75, resize.

Fri Oct 30. New: Top K Frequent Elements (347): HashMap plus min-heap PriorityQueue of size k, then bucket sort for O(n). Revisit: 49. Java: HashMap internals 2: treeification (bucket length 8 and table size at least 64), resize splitting, null key, not thread-safe; HashMap vs Hashtable vs ConcurrentHashMap.

Sat Oct 31 (5 to 6 h). New: Product of Array Except Self (238): prefix and suffix products, no division. Revisit, timed 20 min each from scratch: 217, 1, 49. Java: String immutability, string pool, StringBuilder vs StringBuffer, == vs equals, intern. Career: write 5 one-line stories from your Nordea and Keylane work (biggest migration, hardest bug, a deadline, a conflict, something you owned end to end). You need these for the resume next week.

Sun Nov 1. New: Valid Sudoku (36): HashSet per row, column and box; box index = (r/3)*3 + c/3. Revisit: 347, 238. Java: Q&A Set 1 (Appendix A), answered aloud. Career: write 10 names for your referral list (ex-colleagues now in product companies, I²IT and CDAC seniors, LinkedIn connections at target companies). Then do the weekly review (Appendix D).

Week 2 (Nov 2 to Nov 8): Finish hashing, two pointers, rewrite the resume

Goal: Arrays and Hashing (9) and Two Pointers (5) complete.

Mon Nov 2. New: Encode and Decode Strings (271): length-prefix format such as 4#word. Revisit: 36. Java: lambdas, functional interfaces (Function, Predicate, Consumer, Supplier, BiFunction), method references.

Tue Nov 3. New: Longest Consecutive Sequence (128): HashSet, start counting only at sequence starts. Revisit: 271. Java: Streams I: map, filter, reduce, collect; Collectors toList, toMap, groupingBy, counting.

Wed Nov 4. New: Valid Palindrome (125): two pointers, skip non-alphanumerics with Character.isLetterOrDigit. Revisit: 128. Java: Streams II: flatMap, sorted with Comparator.comparing().thenComparing(), Optional, parallel stream pitfalls.

Thu Nov 5. New: Two Sum II (167): sorted input, two pointers, O(1) space. Revisit: 125. Java: records, sealed classes, pattern matching for instanceof and switch, text blocks, var.

Fri Nov 6. New: 3Sum (15): sort, fix one number, two pointers, skip duplicates. Revisit: 167, 1. Java: checked vs unchecked exceptions, try-with-resources, custom exceptions; Java 21 virtual threads overview.

Sat Nov 7 (5 to 6 h). New: Container With Most Water (11): move the pointer at the shorter side. Revisit, timed: 238, 347, 36. Java: generics (wildcards, type erasure), Comparable vs Comparator. Career (2 h): rewrite the resume using Appendix B.

Sun Nov 8. New: Trapping Rain Water (42, hard): first prefix-max and suffix-max arrays, then two pointers in O(1) space. You may read the solution after 30 min. Revisit: 15, 11. Java: Q&A Set 2. Weekly review.

Week 3 (Nov 9 to Nov 15): Sliding window, stack begins, concurrency I

Goal: Sliding Window (6) complete, 2 stack problems, LinkedIn and GitHub polished. 8 new problems.

Mon Nov 9. New: Best Time to Buy and Sell Stock (121): track the minimum price so far. Revisit: 42 (re-code the two-pointer version). Java: thread lifecycle, Runnable vs Callable, join, daemon threads.

Tue Nov 10. New: Longest Substring Without Repeating Characters (3): window with HashSet or last-index map. Revisit: 121. Java: synchronized and intrinsic locks; write a racy counter with 2 threads, then fix it.

Wed Nov 11. New: Longest Repeating Character Replacement (424): window is valid while length minus max frequency is at most k. Revisit: 3. Java: volatile, happens-before, Java Memory Model basics, AtomicInteger and CAS.

Thu Nov 12. New: Permutation in String (567): fixed-size window with two 26-count arrays. Revisit: 424. Java: ReentrantLock, ReadWriteLock, Condition; write a deadlock and find it with jstack.

Fri Nov 13. New: Minimum Window Substring (76, hard): expand and shrink with have-vs-need counters. You may read the solution after 30 min. Revisit: 567. Java: wait and notify, BlockingQueue producer-consumer.

Sat Nov 14 (5 to 6 h). New: Sliding Window Maximum (239): monotonic deque (ArrayDeque of indexes), and Valid Parentheses (20): stack. Revisit, timed: 3, 424, 15. Java: implement producer-consumer twice, with wait/notify and with BlockingQueue. Career: update LinkedIn (headline, About, skills, experience bullets from Appendix B) and write a README for the dsa-java repo.

Sun Nov 15. New: Min Stack (155): two stacks or a stack of pairs. Revisit: 76, 239. Java: Q&A Set 3. Weekly review.

Week 4 (Nov 16 to Nov 22): Stack, binary search begins, concurrency II

Goal: Stack (7) complete, Binary Search started. 8 new problems. Start outreach.

Mon Nov 16. New: Evaluate Reverse Polish Notation (150): pop the right operand first; division truncates toward zero. Revisit: 20, 155. Java: ExecutorService and thread pools; ThreadPoolExecutor parameters (core, max, queue, rejection policy).

Tue Nov 17. New: Daily Temperatures (739): monotonic decreasing stack of indexes. Revisit: 150. Java: Future vs CompletableFuture: supplyAsync, thenApply, thenCompose, thenCombine, allOf, exceptionally.

Wed Nov 18. New: Car Fleet (853): sort by position descending, compute arrival times, count fleets with a stack. Revisit: 739. Java: ConcurrentHashMap (CAS plus per-bin synchronized since Java 8), CopyOnWriteArrayList, ConcurrentLinkedQueue.

Thu Nov 19. New: Generate Parentheses (22): first backtracking problem; track open and close counts. Revisit: 853. Java: CountDownLatch, CyclicBarrier, Semaphore, ThreadLocal.

Fri Nov 20. New: Largest Rectangle in Histogram (84, hard): monotonic increasing stack. You may read the solution after 30 min. Revisit: 22. Java: virtual threads vs platform threads and when they help (blocking IO); Spring Boot 3.2+ property spring.threads.virtual.enabled=true.

Sat Nov 21 (5 to 6 h). New: Binary Search (704) and Search a 2D Matrix (74). Learn the template: lo = 0, hi = n - 1, while lo <= hi, mid = lo + (hi - lo) / 2. Revisit, timed: 739, 84, 76. Java: build a thread-safe bounded blocking queue and a token-bucket rate limiter. Career: send 5 connection messages (Appendix C, message 1).

Sun Nov 22. New: Koko Eating Bananas (875): binary search on the answer; ceiling division. Revisit: 704, 74. Java: Q&A Set 4. Weekly review.

Week 5 (Nov 23 to Nov 29): Binary search, linked list begins, JVM

Goal: Binary Search (7) complete, Linked List started. 9 new problems.

Mon Nov 23. New: Find Minimum in Rotated Sorted Array (153). Revisit: 875. Java: JVM architecture: class loading, heap, thread stacks, metaspace, program counter.

Tue Nov 24. New: Search in Rotated Sorted Array (33). Revisit: 153. Java: heap generations (Eden, survivor, old), minor vs major GC, object promotion.

Wed Nov 25. New: Time Based Key-Value Store (981): HashMap of key to list of (timestamp, value), binary search on timestamps. Revisit: 33. Java: GC algorithms: G1 (default), ZGC, Parallel; stop-the-world pauses; -Xms, -Xmx and key -XX flags.

Thu Nov 26. New: Reverse Linked List (206): iterative, then recursive. Stretch, 30 min max, read-only: Median of Two Sorted Arrays (4); understand the partition idea, do not grind it. Revisit: 981. Java: memory leaks (static collections, unclosed resources, ThreadLocal, listeners) and OutOfMemoryError types.

Fri Nov 27. New: Merge Two Sorted Lists (21): dummy node. Revisit: 206. Java: JIT, tiered compilation, escape analysis (awareness level only).

Sat Nov 28 (5 to 6 h). New: Linked List Cycle (141, Floyd) and Remove Nth Node From End (19, two pointers with a gap). Revisit, timed: 33, 981, 206. Java: run any local Spring Boot app with -Xlog:gc, take a heap dump with jmap, open it in VisualVM or Eclipse MAT. Career: collect 10 Pune job descriptions from tier C companies (banks and fintech tech centers), note the 5 most repeated skills; send 5 more connection messages.

Sun Nov 29. New: Reorder List (143): find middle, reverse second half, merge. Revisit: 141, 19. Java: Q&A Set 5. Weekly review.

Week 6 (Nov 30 to Dec 6): Linked list, Spring internals, checkpoint

Goal: Linked List (11) complete, checkpoint test, mock interview #1. 6 new problems.

Mon Nov 30. New: Add Two Numbers (2). Revisit: 143. Java: Spring IoC and DI: bean scopes, lifecycle (@PostConstruct, BeanPostProcessor), constructor vs field injection, circular dependencies.

Tue Dec 1. New: Find the Duplicate Number (287): Floyd on the array, or binary search on value. Revisit: 2. Java: Spring Boot auto-configuration: @SpringBootApplication, @Conditional annotations, starters, profiles.

Wed Dec 2. New: Copy List with Random Pointer (138): HashMap old to new, then the O(1) interleave method. Revisit: 287. Java: AOP and proxies (JDK vs CGLIB); @Transactional internals: propagation, isolation, rollback rules, self-invocation pitfall.

Thu Dec 3. New: LRU Cache (146): HashMap plus doubly linked list with dummy head and tail; then the LinkedHashMap(accessOrder = true) variant. Revisit: 138. Java: Spring Security filter chain, JWT validation flow, OAuth2 and OIDC with Azure AD; rehearse your production auth bug end to end.

Fri Dec 4. New: Merge k Sorted Lists (23): PriorityQueue of nodes, then divide and conquer. Revisit: 146. Java: JPA and Hibernate: N+1 problem, lazy vs eager, first and second level cache, dirty checking, optimistic vs pessimistic locking.

Sat Dec 5 (5 to 6 h). New: Reverse Nodes in k-Group (25, hard; you may read the solution after 30 min). Revisit, timed: 23, 21, 19. Java: write a 2-minute and a 5-minute pitch of your main project (architecture, your role, one hard bug, one measurable result). Career: mock interview #1 with a friend or colleague (45 min: one medium problem plus 5 questions from Appendix A).

Sun Dec 6. Checkpoint test. Part 1 (60 min, timer on): pick 2 at random from Minimum Size Subarray Sum (209), Next Greater Element I (496), Find First and Last Position in Sorted Array (34), Sort Colors (75), Intersection of Two Linked Lists (160), Palindrome Linked List (234), Valid Palindrome II (680), Top K Frequent Words (692). Part 2 (20 min): answer 10 random questions from Appendix A aloud. Part 3 (30 min): retrospective.

Checkpoint rules

- Both solved in 60 min with correct complexity, at least one with no hints: start Phase 2 (trees, heaps, graphs, backtracking, DP) and begin tier C referral requests in week 8.
- One of two solved: add a consolidation week repeating your weakest 2 patterns, then move on.
- Zero of two: repeat weeks 5 and 6 DSA only for one more week. Check whether you are reading solutions before the 25-minute mark or skipping the from-scratch re-code.

Appendix A: Sunday Java Q&A sets (answer aloud, 2 minutes each)

Set 1, collections and strings: 1) How does HashMap work internally, including collisions and resize? 2) Why override both equals and hashCode? 3) ArrayList vs LinkedList: when is each faster? 4) HashMap vs TreeMap vs LinkedHashMap? 5) Why is String immutable and what is the string pool? 6) Fail-fast vs fail-safe iterators?

Set 2, language features: 1) What is a functional interface? Give an example. 2) map vs flatMap. 3) What do records give you and what can't they do? 4) What are sealed classes for? 5) Checked vs unchecked exceptions: when do you use each? 6) What is type erasure?

Set 3, concurrency I: 1) What is a race condition and how do you prevent it? 2) synchronized vs ReentrantLock. 3) What does volatile guarantee and not guarantee? 4) How does a deadlock arise and how do you avoid it? 5) wait/notify vs BlockingQueue. 6) What is CAS and where is it used?

Set 4, concurrency II: 1) How does ThreadPoolExecutor decide between a new thread and queueing? 2) CompletableFuture chains vs Future.get(). 3) How is ConcurrentHashMap thread-safe? 4) CountDownLatch vs CyclicBarrier. 5) When would you use virtual threads? 6) ThreadLocal and its leak risk in thread pools.

Set 5, JVM: 1) Explain the JVM memory areas. 2) How does generational GC work? 3) G1 vs ZGC. 4) How do you debug an OutOfMemoryError in production? 5) Common causes of Java memory leaks. 6) What does the JIT compiler do?

Set 6, Spring and data (use in the checkpoint): 1) Constructor vs field injection: why constructor? 2) How does @Transactional work and what is the self-invocation pitfall? 3) Describe the Spring Security filter chain. 4) How is a JWT validated and how would you revoke one? 5) What is the N+1 problem and how do you fix it? 6) Optimistic vs pessimistic locking.

Appendix B: Resume bullets and rules

Formula: action verb + what you built or fixed + how (tech) + measurable result. Replace the [brackets] with your real numbers. If you only have estimates, write them as approximate. Do not invent numbers.

- Migrated 100+ classes from Apache POI to FastExcel in [module], cutting Excel export time by [X%] and memory usage by [Y%] across [N] enterprise clients on the Keylane Plexus platform.
- Diagnosed and fixed a production OIDC and Azure AD authentication failure in a Spring Boot 3 service, restoring login for [N] users within [X hours] and adding [tests or monitoring] to prevent recurrence.
- Built and maintained REST APIs with Spring Boot 3, Spring Security (JWT), JPA/Hibernate and SQL Server for the Nordea Life and Pensions account, handling [N] requests per day.
- Automated build and deployment with GitLab CI/CD to Azure Kubernetes Service, reducing release time from [A] to [B].
- Worked on-site in Copenhagen with the client team, owning [feature or fix] and delivering [result] in a Jira-based agile process.

Rules: one page; Jake's template; skills grouped by Languages, Frameworks, Databases, Cloud and DevOps; 4 to 6 bullets per role; education in 2 lines; add LeetCode and GitHub links; no photo or date of birth.

Appendix C: Message templates

1. Connection request (alumni, seniors, ex-colleagues): Hi [Name], I'm Akash, a Java developer at LTIMindtree (I²IT / CDAC alum). I'm preparing to move into a product company and saw you work at [Company] as [Role]. Would you be open to a 15-minute chat about your experience and the interview process? Thanks either way.

2. Referral request (send from week 8, after the chat): Hi [Name], thanks again for the chat. I'm applying for [Role, link or job ID] at [Company]. I have 3 years of Java and Spring Boot backend experience on banking and pension platforms: production auth (JWT, OIDC), Kubernetes deployments, and a large library migration. Would you be comfortable referring me? Resume attached. Happy to share anything else you need.

Appendix D: Sunday weekly review (30 min)

1. How many problems did I solve alone vs with hints?
2. Which 2 patterns still feel shaky?

3. Which mistakes repeated this week?
4. Did I hit my time block on at least 6 of 7 days?
5. What one change will I make next week?